

Assignment 1: Lexical and Syntax Analysis for MiRust

Version: v1.0

1. OBJECTIVES

- Learn how to specify the lexical and syntax rules of a programming language using ANTLR4.
- Understand tokenization, operator precedence, structs, loops, switch statements, and parse tree generation.
- Build a Python testing harness to run the generated lexer and parser.

2. REQUIRED TASKS TO COMPLETE

- **Task 1: Read the language specification carefully**
 - Study the detailed MiRust Specification document in the `specification/language_spec_v1.0.md` or `.pdf`.
 - Understand the syntax and semantics of the MiRust language.
 - Master the lexical and syntax rules.
- **Task 2: Implement the MiRust.g4 file**
 - Complete the ANTLR4 grammar file located in `src/grammar/MiRust.g4`.
 - Define all lexical rules (tokens).
 - Define all parser rules (grammar rules).
 - Handle operator precedence and associativity correctly.
- **Task 3: Write 100 lexer tests and 100 parser tests**
 - **100 test cases for lexer** in `test/test-lexer.py`
 - Test valid and invalid tokens.
 - Test error handling (unclosed strings, illegal escape sequences, etc.).
 - Test edge cases and boundary conditions.
 - **100 test cases for parser** in `test/test-parser.py`
 - Test valid grammar structures.
 - Test syntax errors and error recovery.
 - Test nested structures and complex expressions.

3. LEXICAL SPECIFICATIONS & ERROR HANDLING REQUIREMENTS

For lexical errors, the generated lexer must throw the corresponding exception defined in `src/utils/lexerror.py`. The exceptions and the conditions under which they must be thrown are:

1. `ErrorToken(s, line)` :

- **When to throw:** Thrown when the lexer encounters an unrecognized character (a character that does not match any valid MiRust token rule).
- **Parameters:** `s` is the unrecognized character string (length 1), `line` is the 1-indexed line number where the unrecognized character is found.

2. `UncloseString(s, line)` :

- **When to throw:** Thrown when a string literal starts with a double quote `"` but is not closed before encountering a raw newline character, a raw tab character, or the end of the file (EOF).

- **Parameters:** `s` is the string content starting from the character immediately after the opening double quote up to (but **excluding**) the raw newline, raw tab, or EOF. `line` is the 1-indexed line number where the unclosed string begins.

3. `IllegalEscape(s, line)` :

- **When to throw:** Thrown when an illegal escape sequence is encountered inside a string literal (any backslash `\` followed by a character other than `n`, `t`, `"`, or `\`).
- **Parameters:** `s` is the string content from the character immediately after the opening double quote up to the character immediately following the illegal escape backslash (i.e. including the invalid escape sequence itself, but excluding the opening double quote). `line` is the 1-indexed line number where the string literal begins.

4. `UnclosedComment(line)` :

- **When to throw:** Thrown when a multi-line comment starts with `/*` but is not closed before reaching the end of the file (EOF).
- **Parameters:** `line` is the 1-indexed line number where the unclosed comment begins.

5. `String Literal Lexemes` :

- The lexeme of a valid string literal token does **not** include the surrounding double quotes `""`. For example, the string literal `"abc"` should have the lexeme `abc`.

4. REQUISITES & SETUP

Before starting, ensure you have set up your Python virtual environment and installed the dependencies listed in `requirements.txt` (refer to **Section 2: Environment Setup Instructions** in `README.md`).

- **Generate Lexer & Parser from ANTLR4 Grammar (`.g4`):**

```
python src/run.py gen
```

- Execute Lexer Unit Tests:

```
python src/run.py test-lexer
```

- Execute Parser Unit Tests:

```
python src/run.py test-parser
```

5. PROJECT STRUCTURE

The project directory structure is shared across multiple assignments, starting with `src` and `test` folders at the root of the submission:

```
root/
├── src/
│   ├── grammar/
│   │   ├── MiRust.g4          # Student's ANTLR4 grammar file
│   │   └── (generated files)  # Generated by ANTLR (Lexer, Parser, etc.)
```

```

├─ test/
│   └─ test-lexer.py           # Student's 100 lexer unit tests
│   └─ test-parser.py        # Student's 100 parser unit tests
└─ requirements.txt

```

6. EVALUATION CRITERIA & GRADING

- **Grammar Implementation Correctness & Completeness (100%):** The grade is based entirely on the accuracy, correctness, and completeness of the generated `MiRust.g4` grammar file. The grammar file must compile without errors or warnings and correctly parse valid programs while rejecting invalid ones according to the specification.
- **Lexical Error Handling (Pass/Fail Gate):** Proper handling of lexical errors matching the Lexical Error Handling Requirements is required to pass the assignment.
- **Test Cases Penalty:** Writing 100 lexer unit tests (in `test/test-lexer.py`) and 100 parser unit tests (in `test/test-parser.py`) is mandatory. While test coverage does not add positive points to the 100% core implementation grade, a penalty of 10% of the total assignment grade will be deducted for each missing test file (up to 20% deduction in total if both are missing).

Lexer Test Distribution Table

The 100 tests in `test/test-lexer.py` must cover the following categories:

Category	Tests	Note
Keywords & Identifiers	test_101 - test_110	Validating language keywords vs identifiers, valid names with underscores, and shadowing.
Valid Integers & Floats	test_111 - test_120	Signed/unsigned integer and float literals, scientific notation exponents (e, E).
Valid Strings	test_121 - test_130	String literals containing valid escape sequences (<code>\n</code> , <code>\t</code> , <code>\"</code> , <code>\\</code>) and special symbols.
Unclosed Strings	test_131 - test_140	Error handling for unclosed string literals (EOF, newline, raw tab).
Illegal Escapes	test_141 - test_150	Error handling for illegal escape sequences inside strings.
Error Tokens	test_151 - test_160	Encountering unrecognized characters (e.g. <code>@</code> , <code>#</code> , <code>\$</code> , <code>?</code> , <code>`</code>).
Operators & Punctuators	test_161 - test_170	Validating arithmetic, comparison, logical operators, range, arrow, punctuation symbols.
Comments & Whitespaces	test_171 - test_175	Single-line comments (<code>//</code>), closed block comments (<code>/* ... */</code>), whitespace skipping.
Unclosed Comments	test_176 - test_180	Error handling for unclosed multi-line comments reaching EOF.
Mixed Streams	test_181 - test_190	Tokenizing full statements and complex expressions with mixed symbols.

Tricky Cases	test_191 - test_200	Advanced edge cases: confusing floats (1.2e+3.4), invalid numbers (123ee4), multiple strings.
---------------------	------------------------	---

Parser Test Distribution Table

The 100 tests in `test/test-parser.py` must cover the following categories:

Category	Tests	Note
Struct Declarations	test_201 - test_210	Validating struct declaration errors (trailing commas, missing colons/braces, semicolon separators, empty structures).
Function Declarations	test_211 - test_225	Validating function definition syntax (missing names/braces/parentheses, wrong parameters, invalid nested structures).
Variable and Type Declarations	test_226 - test_245	Validating variable assignment syntax and types (<code>let/mut</code> keywords, missing names/semicolons, type inference, array sizes).
Control Flow Structures	test_246 - test_260	Syntax checks for <code>if-else</code> conditionals, <code>while</code> loops, range-based for loops, and <code>break/continue/return</code> statements.
Switch Statements	test_261 - test_270	Syntax parser verification of switch-case branching constructs, case clauses, default blocks, and braces.
Expression Precedences	test_271 - test_285	Validating arithmetic, logical negation, relational comparisons, parenthesized expressions, and operator sequencing.
Primary & Complex Accesses	test_286 - test_295	Array literals and index access, struct field accesses, method call arguments, and struct initialization.
Integrated Programs	test_296 - test_300	Syntax check on full program layouts (global vars, invalid declaration orders, nested definitions).

7. DELIVERABLES

Students must submit their code repository preserving the exact directory structure:

- File `MiRust.g4` located in `src/grammar/`.
- File `test-lexer.py` containing 100 lexer tests under `test/`.
- File `test-parser.py` containing 100 parser tests under `test/`.

8. DEADLINE

- **Submission Deadline:** Please refer to the [deadlines.json](#) Gist file for the exact deadline of this assignment.

9. CHANGELOG

- First release under v1.0.